

MPU6050 Demos

Filters & sensor-fusion source for the MPU6050 IMU.

Five sketches: raw readout + bias measurement (00), basic filters (01), complementary filter (02), Madgwick AHRS (03), 1-D Kalman pitch (04).

Rendered 18 Sep 2026 from source in this directory.

00_base_noise.ino

00_base_noise/00_base_noise.ino — 106 lines, 3504 bytes

```
// 00_base_noise.ino – raw 6-axis readout + bias/noise/quantization demo.
//
// Wiring: MPU6050 VCC->3V3, GND->GND, SDA->GPIO8, SCL->GPIO9 (C3 default I2C).
// Scales at defaults: accel +-2g (16384 LSB/g), gyro +-250dps (131 LSB/dps).
// Boot-printed stats are one-time: hold the board perfectly still.
//
// Physics: at rest the gyro's mean IS its bias (small nonzero residual);
// its 1-sigma spread is the noise floor. The accel Z-mean sits near +1g
// ("DC gravity" – not a bias), X/Y means near 0 are real biases.

#include <Wire.h>

#define SDA 8
#define SCL 9
#define MPU 0x68
#define PWR_MGMT_1 0x6B
#define ACCEL_OUT 0x3B

#define NSAMPLES 250 // statistics sample count

const float LSBDPS = 1.0f / 131.0f; // gyro LSB in deg/s
const float LSBG = 1.0f / 16384.0f; // accel LSB in g

int16_t v[7]; // 0-2 accel XYZ, 3 temp, 4-6 gyro XYZ

// One repeated-start transaction reads all 7 registers in 14 bytes.
bool readAll() {
    Wire.beginTransaction(MPU);
    Wire.write(ACCEL_OUT); // registers auto-increment
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom(MPU, 14) != 14) return false;
    for (int i = 0; i < 7; i++) v[i] = (Wire.read() << 8) | Wire.read();
    return true;
}

// int64 sums so sumsq (max ~250 * 2^30) can't overflow; variance in double.
int64_t sumA[3] = { 0 }, sqA[3] = { 0 };
int64_t sumG[3] = { 0 }, sqG[3] = { 0 };

void collectStats() {
    int n = 0, fails = 0;
    while (n < NSAMPLES) {
        if (!readAll()) {
            if (++fails > NSAMPLES) {
                Serial.println("I2C dead – check wiring/power");
                while (1)
                    ;
            }
            delay(5);
            continue;
        }
        for (int i = 0; i < 3; i++) {
            sumA[i] += v[i];
            sqA[i] += (int64_t)v[i] * v[i];
            sumG[i] += v[i + 4];
            sqG[i] += (int64_t)v[i + 4] * v[i + 4];
        }
        n++;
    }

    Serial.println("axis  bias      1-sigma  (accel g / gyro dps)");

    for (int i = 0; i < 3; i++) {
        double mA = (double)sumA[i] / NSAMPLES;
        double sA = sqrt(((double)sqA[i] / NSAMPLES - mA * mA);
        double mG = (double)sumG[i] / NSAMPLES;
        double sG = sqrt(((double)sqG[i] / NSAMPLES - mG * mG);
```

```

    Serial.printf("A%c  %+8.4f  %8.4f g\n", 'X' + i, mA * LSBG, SA * LSBG);
    Serial.printf("G%c  %+8.4f  %8.4f dps\n", 'X' + i, mG * LSBDPS, sG * LSBDPS);
}

// Quantization: 1 LSB in physical units. If 1-sigma / LSB ~ 1.2 the ADC is
// quantization-limited; if the ratio is large, white noise dominates and
// quantization is invisible – the usual case for the 6050's noise.
double m = (double)sumG[0] / NSAMPLES;
double sdG_x = sqrt((double)sqG[0] / NSAMPLES - m * m);
Serial.printf("quantization: acc 1 LSB = %.3f mg | gyro 1 LSB = %.4f dps\n",
    1.0f / 16384.0f * 1000.0f, 1.0f / 131.0f);
Serial.printf("gyroX noise / LSB = %.1f -> %s\n",
    sdG_x,
    (sdG_x < 2.0) ? "quantization-limited" : "noise-dominated");
}

void setup() {
    Serial.begin(115200);
    Wire.begin(SDA, SCL);
    Wire.setClock(100000); // 400kHz fails on many 6050 clones -> clock-stretch
    Wire.beginTransmission(MPU);
    Wire.write(PWR_MGMT_1);
    Wire.write(0);
    Wire.endTransmission(); // wake: clear SLEEP in PWR_MGMT_1
    delay(50);
    collectStats();
}

void loop() {
    if (!readAll()) {
        Serial.println("I2C fail");
        delay(500);
        return;
    }
    float tempC = v[3] / 340.0f + 36.53f;
    Serial.printf("a %6d %6d %6d | g %6d %6d %6d | %.2f C\n",
        v[0], v[1], v[2], v[4], v[5], v[6], tempC);
    delay(100);
}

```

01_filters.ino

01_filters/01_filters.ino — 66 lines, 1807 bytes

```
#include <Wire.h>
#include <math.h>

#define SDA 8
#define SCL 9
#define MPU 0x68
#define PWR_MGMT_1 0x6B
#define ACCEL_OUT 0x3B

#define FC_LP_ACC 6.0f // Hz, accel smoothing
#define FC_HP_GYR 0.1f // Hz, gyro drift removal

constexpr float LSB DPS = 1.0f / 131.0f;
constexpr float LSB G = 1.0f / 16384.0f;

int16_t v[7]; // 0-2 accel XYZ, 3 temp, 4-6 gyro XYZ

// Filter states
float aZlp = 0.0f;
float gYhp = 0.0f;
float gY_prev = 0.0f;

// One repeated-start read: all 7 registers in 14 bytes (see 00).
bool readAll() {
    Wire.beginTransmission(MPU);
    Wire.write(ACCEL_OUT);
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom(MPU, 14) != 14) return false;
    for (int i = 0; i < 7; i++) v[i] = (Wire.read() < 8) | Wire.read();
    return true;
}

void setup() {
    Serial.begin(115200);
    Wire.begin(SDA, SCL);
    Wire.setClock(100000);
    Wire.beginTransmission(MPU); Wire.write(PWR_MGMT_1); Wire.write(0);
    Wire.endTransmission(); // wake
    delay(50);
}

void loop() {
    if (!readAll()) { Serial.println("I2C fail"); delay(500); return; }

    static uint32_t tPrev = 0;
    uint32_t tNow = millis();
    float dt = (tPrev ? (tNow - tPrev) : 20) / 1000.0f;
    tPrev = tNow;
    if (dt > 0.25f) dt = 0.02f; // after a long stall, don't spike the filters

    // Direct coefficient calculation without structs
    float aLP = dt / ((1.0f / (2.0f * PI * FC_LP_ACC)) + dt);
    float aHP = (1.0f / (2.0f * PI * FC_HP_GYR)) / ((1.0f / (2.0f * PI * FC_HP_GYR)) + dt);

    float aZ = v[2] * LSB G; // g
    float gY = v[5] * LSB DPS; // deg/s

    // Inline filter steps
    aZlp += aLP * (aZ - aZlp);
    gYhp = aHP * (gYhp + gY - gY_prev);
    gY_prev = gY;

    Serial.printf("accZ raw %+7.3f lp %+7.3f | gyroY raw %+7.3f hp %+7.3f deg/s\n",
                  aZ, aZlp, gY, gYhp);
    delay(20);
}
```

02_complementary.ino

02_complementary/02_complementary.ino — 71 lines, 2073 bytes

```
#include <Wire.h>
#include <math.h>

#define SDA 8
#define SCL 9
#define MPU 0x68
#define PWR_MGMT_1 0x6B
#define ACCEL_OUT 0x3B

#define TAU 0.5f          // Crossover time in seconds
#define LOOP_MS 20

constexpr float LSBDPS = 1.0f / 131.0f;
constexpr float RAD2DEG = 180.0f / PI;

int16_t v[7];    // 0-2 accel XYZ, 3 temp, 4-6 gyro XYZ

// Raw integrated gyro angles (unfiltered dead-reckoning)
float gyroRoll = 0.0f, gyroPitch = 0.0f, gyroYaw = 0.0f;

// Complementary filter outputs
float fusedRoll = 0.0f, fusedPitch = 0.0f, fusedYaw = 0.0f;

bool readAll() {
    Wire.beginTransmission(MPU);
    Wire.write(ACCEL_OUT);
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom(MPU, 14) != 14) return false;
    for (int i = 0; i < 7; i++) v[i] = (Wire.read() <= 8) | Wire.read();
    return true;
}

void setup() {
    Serial.begin(115200);
    Wire.begin(SDA, SCL);
    Wire.setClock(100000);
    Wire.beginTransmission(MPU); Wire.write(PWR_MGMT_1); Wire.write(0);
    Wire.endTransmission();    // wake
    delay(50);
}

void loop() {
    if (!readAll()) { Serial.println("I2C fail"); delay(500); return; }

    float dt = LOOP_MS / 1000.0f;
    float a = TAU / (TAU + dt);

    // Raw gyro rates (deg/s)
    float rollRate = v[4] * LSBDPS;
    float pitchRate = v[5] * LSBDPS;
    float yawRate = v[6] * LSBDPS;

    // Unfiltered raw gyro dead-reckoning (pure integration)
    gyroRoll += rollRate * dt;
    gyroPitch += pitchRate * dt;
    gyroYaw += yawRate * dt;

    // Accelerometer reference angles (degrees)
    float accRoll = atan2f(v[1], v[2]) * RAD2DEG;
    float accPitch = atan2f(-v[0], sqrtf((float)v[1] * v[1] + (float)v[2] * v[2])) * RAD2DEG;

    // Complementary fusion
    fusedRoll = a * (fusedRoll + rollRate * dt) + (1.0f - a) * accRoll;
    fusedPitch = a * (fusedPitch + pitchRate * dt) + (1.0f - a) * accPitch;
    fusedYaw = gyroYaw; // No magnetometer

    Serial.printf("RAW_GYRO -> R:%+6.1f P:%+6.1f Y:%+6.1f | FUSED -> R:%+6.1f P:%+6.1f Y:%+6.1f\n",
```

```
        gyroRoll, gyroPitch, gyroYaw, fusedRoll, fusedPitch, fusedYaw);  
    delay(LOOP_MS);  
}
```

03_madgwick.ino

03_madgwick/03_madgwick.ino — 111 lines, 4314 bytes

```
// 03_madgwick.ino – full 3-axis orientation (roll/pitch/yaw) via Madgwick's
// gradient-descent IMU filter. No assumption of a single tilt axis.
//
// Wiring: MPU6050 VCC->3V3, GND->GND, SDA->GPIO8, SCL->GPIO9 (C3 default I2C).
// Scales at defaults: accel +-2g (16384 LSB/g), gyro +-250dps (131 LSB/dps).
//
// Algorithm (SOH Madgwick, x-io "open-source IMU and AHRS") – two fused steps:
// 1) quaternion_dot = 0.5*q (x) gyro_rad -> short-term integration
// 2) gradient-direction correction of that dot toward the direction that
//    lets the quaternion reproduce the measured gravity (accelerometer),
//    with gain beta = gyro measurement error in rad/s. beta too high ->
//    accelerometer noise bleeds through; too low -> gyro drift wins.
// yaw is UNREFERENCED without a magnetometer: it integrates gyro drift.

#include <Wire.h>
#include <math.h>

#define SDA 8
#define SCL 9
#define MPU 0x68
#define PWR_MGMT_1 0x6B
#define ACCEL_OUT 0x3B
#define LOOP_MS 20

#define BETA 0.1f // gyro-measurement-error gain, rad/s

constexpr float LSBDPS = 1.0f / 131.0f;
constexpr float DEG2RAD = PI / 180.0f;
constexpr float RAD2DEG = 180.0f / PI;

int16_t v[7]; // 0-2 accel XYZ, 3 temp, 4-6 gyro XYZ

// scalar-first quaternion state
float q0 = 1.0f, q1 = 0.0f, q2 = 0.0f, q3 = 0.0f;

// Transcribed from Arduino-libraries/MadgwickAHRS updateIMU() (GPL), with dt
// measured live instead of a fixed invSampleFreq. Verified rows s1/s2 use az
// (the Jacobian's third row), not ax -- a common transcription error.
void madgwickIMU(float gx, float gy, float gz, // deg/s
                 float ax, float ay, float az, // g, any scale
                 float dt) {
    gx *= DEG2RAD; gy *= DEG2RAD; gz *= DEG2RAD; // Arduino-lib converts inside

    float qDot1 = 0.5f * (-q1 * gx - q2 * gy - q3 * gz);
    float qDot2 = 0.5f * ( q0 * gx + q2 * gz - q3 * gy);
    float qDot3 = 0.5f * ( q0 * gy - q1 * gz + q3 * gx);
    float qDot4 = 0.5f * ( q0 * gz + q1 * gy - q2 * gx);

    if (!(ax == 0.0f && ay == 0.0f && az == 0.0f)) { // guard NaN in normalize
        float recipNorm = 1.0f / sqrtf(ax * ax + ay * ay + az * az);
        ax *= recipNorm; ay *= recipNorm; az *= recipNorm;

        float _2q0 = 2*q0, _2q1 = 2*q1, _2q2 = 2*q2, _2q3 = 2*q3;
        float _4q0 = 4*q0, _4q1 = 4*q1, _4q2 = 4*q2;
        float _8q1 = 8*q1, _8q2 = 8*q2;
        float q0q0 = q0*q0, q1q1 = q1*q1, q2q2 = q2*q2, q3q3 = q3*q3;

        // gradient of |quaternion->gravity error|, solved least-squares inline
        float s0 = _4q0*q2q2 + _2q2*ax + _4q0*q1q1 - _2q1*ay;
        float s1 = _4q1*q3q3 - _2q3*ax + 4*q0q0*q1 - _2q0*ay - _4q1 + _8q1*q1q1 + _8q1*q2q2 + _4q1*az;
        float s2 = 4*q0q0*q2 + _2q0*ax + _4q2*q3q3 - _2q3*ay - _4q2 + _8q2*q1q1 + _8q2*q2q2 + _4q2*az;
        float s3 = 4*q1q1*q3 - _2q1*ax + 4*q2q2*q3 - _2q2*ay;

        recipNorm = 1.0f / sqrtf(s0*s0 + s1*s1 + s2*s2 + s3*s3);
        s0 *= recipNorm; s1 *= recipNorm; s2 *= recipNorm; s3 *= recipNorm;

        qDot1 -= BETA * s0;
```

```

    qDot2 -= BETA * s1;
    qDot3 -= BETA * s2;
    qDot4 -= BETA * s3;
}

q0 += qDot1 * dt; q1 += qDot2 * dt; q2 += qDot3 * dt; q3 += qDot4 * dt;

float r = 1.0f / sqrtf(q0*q0 + q1*q1 + q2*q2 + q3*q3);    // keep |q| = 1
q0 *= r; q1 *= r; q2 *= r; q3 *= r;
}

bool readAll() {
    Wire.beginTransaction(MPU);
    Wire.write(ACCEL_OUT);
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom(MPU, 14) != 14) return false;
    for (int i = 0; i < 7; i++) v[i] = (Wire.read() << 8) | Wire.read();
    return true;
}

void setup() {
    Serial.begin(115200);
    Wire.begin(SDA, SCL);
    Wire.setClock(100000);
    Wire.beginTransaction(MPU); Wire.write(PWR_MGMT_1); Wire.write(0);
    Wire.endTransmission();    // wake
    delay(50);
}

void loop() {
    if (!readAll()) { Serial.println("I2C fail"); delay(500); return; }

    madgwickIMU(v[4] * LSB_DPS, v[5] * LSB_DPS, v[6] * LSB_DPS,
                v[0], v[1], v[2], LOOP_MS / 1000.0f);

    // canonical Euler extraction from scalar-first quaternion
    float roll = atan2f(q0*q1 + q2*q3, 0.5f - q1*q1 - q2*q2) * RAD2DEG;
    float pitch = asinf(-2.0f * (q1*q3 - q0*q2)) * RAD2DEG;
    float yaw = atan2f(q1*q2 + q0*q3, 0.5f - q2*q2 - q3*q3) * RAD2DEG;

    Serial.printf("roll %+6.1f pitch %+6.1f yaw %+7.1f (yaw drifts: no magnetometer)\n",
                  roll, pitch, yaw);
    delay(LOOP_MS);
}

```

04_kalman_1d_pitch.ino

04_kalman_1d_pitch/04_kalman_1d_pitch.ino — 142 lines, 4277 bytes

```
#include <Wire.h>
#include <math.h>

#define SDA_PIN 8
#define SCL_PIN 9
constexpr uint8_t MPU_ADDR = 0x68;

constexpr float ACCEL_SCALE = 16384.0f; // ±2g
constexpr float GYRO_SCALE = 131.0f; // ±250 deg/s
constexpr float RAD2DEG = 180.0f / PI;

// -----
// Kalman Filter State & Noise Parameters (All in deg, deg/s)
// -----
float kalmanAngle = 0.0f;
float kalmanBias = 0.0f;
float P00 = 4.0f, P01 = 0.0f, P11 = 0.25f; // Symmetric covariance (P10 == P01)

const float Q_GYRO = 0.02f * 0.02f; // Gyro rate noise variance [(deg/s)^2]
const float Q_BIAS = 0.001f * 0.001f; // Bias random-walk variance [(deg/s)^2]
const float R_ACC = 1.0f; // Accel measurement variance [deg^2]

// Raw gyro dead-reckoning accumulator
float rawGyroAngle = 0.0f;
uint32_t lastMicros = 0;

// -----
// MPU6050 Low-Level Helpers
// -----
void writeRegister(uint8_t reg, uint8_t val) {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(reg);
    Wire.write(val);
    Wire.endTransmission();
}

bool readSensors(float &ax_g, float &ay_g, float &az_g, float &gyroY_dps) {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x3B); // ACCEL_XOUT_H
    if (Wire.endTransmission(false) != 0 || Wire.requestFrom(MPU_ADDR, (uint8_t)14) != 14) {
        return false;
    }

    int16_t ax = (Wire.read() << 8) | Wire.read();
    int16_t ay = (Wire.read() << 8) | Wire.read();
    int16_t az = (Wire.read() << 8) | Wire.read();
    Wire.read(); Wire.read(); // Skip temp
    Wire.read(); Wire.read(); // Skip gyro X
    int16_t gy = (Wire.read() << 8) | Wire.read();
    Wire.read(); Wire.read(); // Skip gyro Z

    ax_g = (float)ax / ACCEL_SCALE;
    ay_g = (float)ay / ACCEL_SCALE;
    az_g = (float)az / ACCEL_SCALE;
    gyroY_dps = (float)gy / GYRO_SCALE;
    return true;
}

float getAccelPitch(float ax, float ay, float az) {
    return atan2f(-ax, sqrtf(ay * ay + az * az)) * RAD2DEG;
}

// -----
// Setup
// -----
void setup() {
    Serial.begin(115200);
```

```

Wire.begin(SDA_PIN, SCL_PIN);
Wire.setClock(400000);
delay(100);

writeRegister(0x6B, 0x00); // Wake MPU6050
writeRegister(0x1A, 0x03); // DLPF ~44Hz
writeRegister(0x1B, 0x00); // Gyro ±250 dps
writeRegister(0x1C, 0x00); // Accel ±2g
writeRegister(0x19, 0x04); // 200 Hz sample rate divider

delay(100);

// Quick initial calibration while stationary
Serial.println("Calibrating stationary gyro bias...");
double gyroSum = 0.0;
float ax, ay, az, gy;
for (int i = 0; i < 500; ++i) {
    if (readSensors(ax, ay, az, gy)) gyroSum += gy;
    delay(2);
}
kalmanBias = (float)(gyroSum / 500.0);

// Prime initial angle from gravity
readSensors(ax, ay, az, gy);
kalmanAngle = getAccelPitch(ax, ay, az);
rawGyroAngle = kalmanAngle;

lastMicros = micros();
Serial.println("System initialized.");
}

// -----
// Loop
// -----
void loop() {
    float ax, ay, az, gyroY;
    if (!readSensors(ax, ay, az, gyroY)) return;

    uint32_t now = micros();
    float dt = (now - lastMicros) * 1e-6f;
    lastMicros = now;
    if (dt <= 0.0f || dt > 0.05f) dt = 0.005f; // Clamped to 200 Hz fallback

    // Unfiltered dead reckoning for baseline comparison
    rawGyroAngle += (gyroY - kalmanBias) * dt;

    // 1. Accel Pitch Reference
    float accelPitch = getAccelPitch(ax, ay, az);

    // 2. Kalman Prediction Step
    kalmanAngle += (gyroY - kalmanBias) * dt;
    P00 += dt * (dt * P11 - 2.0f * P01 + Q_GYRO);
    P01 -= dt * P11;
    P11 += Q_BIAS;

    // 3. Kalman Update Step
    float S = P00 + R_ACC;
    float K0 = P00 / S;
    float K1 = P01 / S;
    float innov = accelPitch - kalmanAngle;

    kalmanAngle += K0 * innov;
    kalmanBias += K1 * innov;

    P00 -= K0 * P00;
    P01 -= K0 * P01;
    P11 -= K1 * P01;

    // Formatted tabular print
    Serial.printf("RAW_GYR: %+6.2f | ACCEL: %+6.2f | KALMAN: %+6.2f | BIAS: %+7.4f deg/s\n",
        rawGyroAngle, accelPitch, kalmanAngle, kalmanBias);
}

```

```
    delay(2);  
}
```